CS 410 Compiler Construction Phase 2 Parser

Expectations

As a team, review your deliverables from the previous phase and update them only when necessary. Then…

A **Create a context-free grammar** for your minimal dialect.
- Generate the syntax of your dialect using necessary and sufficient productions, nonterminals, and terminals.
  - Represent nonterminals in upper-case or title-case without bracketing and terminals in lower-case without quoting.
  - Include the constructs of your dialect as the productions and nonterminals of your grammar.
  - Include the tokens of your dialect accepted by your scanner as the terminals of your grammar.
  - Exclude any productions replicating lexical analysis, such as processing individual letters or digits.
- Follow one of these grammar properties: simple, quasi-simple, or LL(1) / LR(1).

B **Define an abstract machine** for syntactic analysis of grammar from step A.
- Follow one of these parser models:

Model	Representation	Strategy	Grammar	Hint
Pushdown machine parser	State transition table(s)	Top down	LL(1) or simpler	Try simple grammar
Recursive descent parser	Pseudocode functions	Top down	LL(1) or simpler	Avoid left recursion
Shift reduce parser	Action table, goto table	Bottom up	LR(1) or simpler	Try elegant grammar

- Accept the valid sentences of your dialect. Reject any invalid sentences.
- Follow the conventional notation for your parser model and representation.

C **Program a parser** in hand-written Java based on your abstract machine from step B.
- Follow either your own design or a recommended design, using data structures appropriate for your parser model.
- Input a stream of tokens from the output of your scanner or from a file, user, or standard input channel.
- Translate your source code tokens into these intermediate code atoms with corresponding semantics:

Class	**Atom** with Operation, Operands, & Placeholders	**Semantics**
1	`(ADD, left, right, result,  ,  )`	Let result = left + right
2	`(SUB, left, right, result,  ,  )`	Let result = left – right
3	`(MUL, left, right, result,  ,  )`	Let result = left × right
4	`(DIV, left, right, result,  ,  )`	Let result = left ÷ right
5	`(JMP,  ,  ,  ,  , dest)`	Goto destination label unconditionally
10	`(NEG, left,  , result,  ,  )`	Let result = – left
11	`(LBL,  ,  ,  ,  , dest)`	Label for goto destination
12	`(TST, left, right,  , comp, dest)`	Goto destination label conditionally when comparison between left and right is true as defined from 1 to 6…

1	2	3	4	5	6
equal	lesser	greater	lesser or equal	greater or equal	unequal

13	`(MOV, left,  , result,  ,  )`	Let result = left

- For more examples and non-examples of notation for atoms, see Appendix 1.
- Output a stream of atoms to a file, user, or standard output channel (one per line or entry).
- Make this with hand-written Java, not with a compiler-compiler (reserved for a future assignment).

Assessment

Submit your deliverables on eCampus by the deadline.

- **Create and attach your credits** with expected vs actual contributions and effort percentage for each member (1 page).
- If applicable, **attach any updated deliverables** from the previous phase (formats vary accordingly).
- **Attach your context-free grammar** as an outline or spreadsheet (1 or 2 pages).
- **Attach your abstract machine** as a hand-drawn illustration (1 or 2 pages).
- **Attach your parser** as hand-written Java source code (1 zipped project folder).

Demonstrate your executables to the instructor by the deadline or when arranged (submit your deliverables before, if possible).

Appendix 1

Notation	**Examples** (valid)	**Non-Examples** (invalid)

Casing of operation is permissive,
but naming of operation is strict.
	`MOV`
	`Mov`
	`mov`
		`MOVE`
		`LET`
		`=`

Whitespace is permissive,
but enclosures and delimiters are strict.
	`( MOV , left , , result , , )`
	`(MOV, left, result, , )`
	`(MOV,left,,result,,)`
		`(MOV|left||result||)`
		`[MOV,left,,result,,]`
		`MOV left result`

Trailing placeholders are permissive,
but leading placeholders and order of
operation and operands are strict.
	`(MOV, left, _, result, _, _)`
	`(MOV, left, _, result, _)`
	`(MOV, left, _, result)`
		`(MOV, left, _, _, _, result)`
		`(MOV, left, result, _, _, _)`
		`(_, _, _, result, left, MOV)`